

Étude de cas — Le paradoxe du point aveugle « Blind Spot »

Filière Recherche · 3e année · Groupe de 4 à 5 étudiants · 2 semaines

1. Contexte: la course entre adaptation et détection

Un modèle de machine learning déployé en production apprend sur des données qui changent. En finance, un modèle calibré sur un marché calme continue de produire des signaux quand le marché devient turbulent — sauf qu'ils sont devenus faux. Ce changement temporel de la relation conditionnelle $P(y | x)$ entre les variables d'entrée x et la cible y s'appelle un **concept drift** (Gama et al., 2014). Il se distingue du **data drift**, qui ne concerne que l'évolution de la distribution des entrées $P(x)$ (Moreno-Torres et al., 2012).

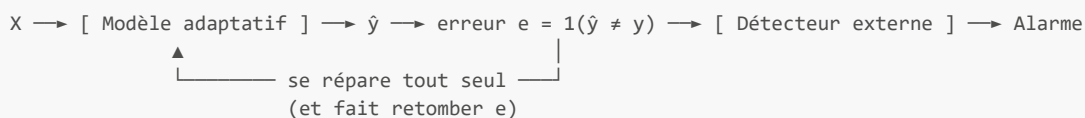
Cette distinction modifie radicalement l'architecture de surveillance.

DATA DRIFT — le détecteur regarde les entrées



Pas besoin d'étiquettes. Pas de boucle : le détecteur ignore le modèle.

CONCEPT DRIFT — le détecteur regarde les erreurs du modèle



Il faut les étiquettes y . Et il y a une boucle.

C'est cette boucle qui pose problème.

L'architecture standard en flux de données associe deux briques. Un **classifieur adaptatif**, qui se répare tout seul : l'**ARF** est une forêt d'arbres où chaque arbre possède son propre détecteur interne à fenêtre adaptative, **ADWIN**, et se fait remplacer quand il se dégrade. Et un **détecteur externe cumulatif** comme **CUSUM** / **Page-Hinkley**, qui accumule l'excès d'erreur au fil du temps et alarme quand le cumul dépasse un seuil.

La première brique répare. La seconde prévient l'humain : elle déclenche un réentraînement, un audit, une mise en pause de la stratégie.

Le problème. L'ARF se répare continuellement, ce qui fait redescendre son taux d'erreur. Si cette réparation est plus rapide que le temps d'accumulation de la preuve par le détecteur externe, le signal d'erreur disparaît avant que l'alarme ne se déclenche. Le modèle va bien, l'opérateur n'a rien vu, et personne ne sait que le monde a changé. C'est le *paradoxe du point aveugle*.

Contre-intuitivement, **plus le drift est violent, plus il risque de passer inaperçu** : un gros choc déclenche une réparation plus rapide, donc un signal plus bref.

C'est ce phénomène que décrit l'article *The Blind Spot Paradox: When Adaptive Classifiers Defeat Drift Detectors*, soumis à IEEE ICDM 2026.

2. Etat de l'article de référence

Quatre experts internationaux l'ont évalué. Ils reconnaissent l'intérêt du phénomène et la qualité du dispositif expérimental — le code est public, les expériences sont rejouables. Ils pointent en revanche des faiblesses dans la partie théorique. L'une d'elles, une simplification dans une majoration de probabilité, a déjà été corrigée : c'est la version **v64** que vous allez lire.

Il reste la partie que nous vous confions, et qui est la plus ouverte : décrire correctement la **course entre le classifieur et le détecteur** quand le classifieur est une forêt.

3. Un enjeu majeur en Finance

Le phénomène est général — il apparaîtrait sur n'importe quel flux. Il a été découvert sur des données financières, pour une raison simple : ce sont les séries où le drift est à la fois le plus fréquent, le plus brutal et le plus coûteux. En salle de marché, identifier les concept drifts, ou « changements

de régime », constitue un défi majeur : lorsqu'un choc macroéconomique se produit, les dynamiques des prix de nombreux actifs et les relations entre ces actifs peuvent être profondément affectées.

Un exemple concret. On cherche à prédire le signe du rendement du CAC 40 le lendemain. La variable cible y vaut 1 si le rendement est positif, 0 sinon. Les variables explicatives X sont les rendements des cinq derniers jours, la volatilité réalisée sur vingt jours, le volume échangé et un indice de volatilité implicite.

En régime calme, les tendances persistent : une série de hausses récentes annonce plutôt une hausse. Le modèle apprend cette relation. Survient un choc — annonce macroéconomique, tension géopolitique — et le marché bascule en régime de stress. Les mêmes valeurs de X ne prédisent plus le même y : le momentum s'inverse, les hausses récentes annoncent désormais des corrections brutales. La loi $P(y | X)$ a changé alors que X peut très bien avoir gardé la même allure. C'est un **concept drift**. Si seule la volatilité de X avait augmenté sans que la relation change, on aurait un simple **data drift**, sans conséquence sur la qualité des prédictions.

Trois caractéristiques rendent ces séries difficiles :

1. **Les régimes** : un marché alterne des phases calmes et des phases turbulentes, et chaque bascule est un drift.
2. **L'hétéroscédasticité** : la volatilité se regroupe en paquets, et un détecteur réglé sur une phase calme se met à alarmer en continu dès qu'elle monte.
3. **L'absence de vérité terrain** : sur des données de marché réelles, il est difficile voire impossible de déterminer à quel instant exact le régime a changé, donc personne ne peut mesurer si un détecteur a raison.

C'est pour cette dernière raison que l'article utilise **ProteuS**, un générateur de flux financiers synthétiques qui reproduit l'hétéroscédasticité et les changements de régime **avec des instants de rupture connus**. On peut donc compter les détections manquées et les fausses alarmes.

On a alors besoin de l'un ou de l'autre — un modèle prédictif robuste au concept drift par construction, ou un objectif intermédiaire : un détecteur fiable qui dise quand recalibrer nos modèles, qu'il s'agisse de modèles prédictifs des prix ou de modèles de pricing de produits financiers « dérivés ». L'architecture standard vise le second. Paradoxalement, c'est le bon fonctionnement du classifieur adaptatif qui casse la surveillance : en se réparant, il efface lui-même la preuve que le détecteur attendait.

Le résultat obtenu sur ProteuS est le plus parlant de l'article. Pour éviter les fausses alarmes causées par la volatilité, il faut un seuil de détection élevé ($\lambda \geq 15$). Pour attraper le drift avant que la forêt ne l'efface, il faut un seuil bas ($\lambda \leq 12,4$). Les deux contraintes ne se recouvrent pas : aucun réglage ne fonctionne. Un détecteur à fenêtres comme **KSWIN** — qui compare deux fenêtres du flux d'erreur au lieu d'y accumuler de la preuve — résiste mieux sur les réglages testés, sans que son immunité soit établie en général.

4. Votre mission

L'article mesure la vitesse de réparation de la forêt par **le temps écoulé jusqu'au premier arbre remplacé**, noté τ_{ARF} (Définition 2 du manuscrit). Il compare ce temps à celui qu'il faut au détecteur pour alarmer. Deux choses clochent dans cette comparaison. Ce sont vos deux versants.

4.1 Versant théorique — une course entre deux horloges aléatoires

Le manuscrit compare τ_{ARF} , qui est une **variable aléatoire**, à une quantité $\tau_{\text{det}}^* = \lambda / (\Delta e - \delta_P)$, qui est un **nombre fixe** : le temps qu'il faudrait au détecteur pour alarmer si l'erreur se comportait exactement comme sa moyenne (Proposition 9).

Or le détecteur, lui aussi, alarme à un instant aléatoire. Parfois plus tôt, parfois plus tard, voire **jamais**.

Question A. Formalisez la course entre ces deux instants. Que vaut $P(\tau_{\text{ARF}} < \tau_{\text{det}})$?

*Piste : utilisez le cadre statistique des **risques concurrents avec censure** (voir [Fine et Gray, 1999](#)).*

Trois formes de réponse sont recevables, et elles se complètent :

1. **Poser l'objet correctement** : que vaut cette probabilité quand τ_{det} peut valoir l'infini, et pourquoi une exécution où le détecteur n'alarme jamais ne doit surtout pas être écartée du calcul.
2. **Encadrer analytiquement** : il existe des bornes qui ne dépendent que des lois marginales de τ_{ARF} et de τ_{det} , valables quelle que soit leur dépendance ; elles se démontrent en quelques lignes à partir d'inclusions d'événements bien choisies, et elles montrent au passage que les seules marginales ne suffisent pas à déterminer la réponse.
3. **Estimer empiriquement** : par la fonction d'incidence cumulée, sur vos exécutions.

Conseil : Ne cherchez pas une formule fermée. Les deux temps sont couplés par l'adaptation elle-même — c'est le sujet de l'article — et aucune expression close n'est connue. Un encadrement démontré plus une estimation propre valent mieux qu'une formule inventée.

Une seconde difficulté est imbriquée dans la première. La forêt s'adapte dès que **le premier** de ses M arbres se fait remplacer, donc $\tau_{\text{ARF}} = \min(\tau_1, \dots, \tau_M)$. Pour calculer la loi de ce minimum, le manuscrit suppose les arbres **indépendants** et écrit $P(\tau_{\text{ARF}} < s) = 1 - [1 - F(s)]^M$, où F est la fonction de répartition du temps d'un arbre seul. Mais les M arbres voient **le même flux de données**. Ils ne sont pas indépendants.

Question B. L'hypothèse d'indépendance rend-elle la formule théorique de l'article trop optimiste ou pessimiste ? Démontrez-le mathématiquement ou fournissez un contre-exemple.

Dites toujours par rapport à quoi. « Optimiste » et « pessimiste » changent de sens selon la grandeur regardée (et peuvent être vraies simultanément : *exemple: la formule peut surestimer la vitesse d'adaptation de la forêt et, par voie de conséquence, surestimer la probabilité que le détecteur manque le drift*):

1. Répondez d'abord sur la vitesse d'adaptation τ_{ARF} .
2. Puis déduisez-en ce que cela implique pour P_{miss} .

C'est la question la plus intéressante du sujet, et personne ne l'a encore traitée. La réponse du manuscrit peut être soit démontrée soit infirmée. Votre travail est de trancher.

Pistes de départ — à explorer :

- Les [statistiques d'ordre](#) donnent la loi du minimum de variables indépendantes. Que devient le résultat sous dépendance ?
- Que se passe-t-il si l'on **conditionne** par la réalisation du flux ? Sachant le flux, qu'est-ce qui distingue encore un arbre d'un autre dans l'ARF ? Ces différences-là sont-elles indépendantes entre arbres ?
- Une fois le conditionnement posé, une inégalité de convexité classique (Jensen, appliquée à $x \mapsto x^M$ sur $[0,1]$) donne peut-être le sens de l'inégalité recherchée.
- Une course entre deux instants dont l'un peut ne jamais survenir porte un nom en statistique : les [risques concurrents avec censure](#). Ce cadre est standard en analyse de survie et n'a jamais été appliqué à la détection de drift. C'est le seul point du sujet qui demande un vrai travail bibliographique.

*Ne vous laissez pas impressionner : aucun de ces outils ne dépasse le niveau d'un cours de probabilités de deuxième année. La difficulté n'est pas technique, elle est de **poser le bon modèle**.*

4.2 Versant expérimental — mesurer la véritable adaptation

Remplacer un seul arbre ne guérit pas (nécessairement) instantanément la forêt. Si vous changez dix pour cent des modèles, vous n'altérez qu'un dixième des prédictions finales. Il faut donc [instrumenter](#) le code pour ausculter sa vraie dynamique.

Question C. Le temps τ_{ARF} est-il un bon indicateur de l'adaptation réelle de la forêt ? Définissez au moins deux mesures alternatives — par exemple la fraction d'arbres remplacés, et l'instant où l'erreur globale redescend à moins de p de son niveau d'avant le drift.

Quatre options figurent dans cette grille. Sentez-vous libres d'en concevoir d'autres. Lisez impérativement le point d'**ATTENTION** ci-dessous avant d'écrire du code Python.

Indicateur	Définition retenue	Protocole d'évaluation
$\tau_{\text{swap}}(q)$	Le moment où la proportion q d'arbres a été remplacée (pour q valant 10 %, 25 %, 50 % ou 75 %).	Run par run. Le compteur interne de remplacements est un entier.
$\tau_{\text{err}}(p)$	Instant où l'erreur de l'ensemble redescend à moins de p par rapport au socle pré-rupture.	Exclusivement sur la moyenne inter-graines , jamais sur un seul run.
$A(H)$	Cumul des erreurs excédentaires calculé sur une fenêtre fixe de H pas après le drift : $A(H) = \sum (e_t - \hat{p}_0)$, où $\hat{p}_0$ représente le taux d'erreur mesuré avant le drift.	Run par run. On soustrait simplement une constante au signal binaire: aucun lissage nécessaire.
$S_{\text{max}}(H)$	La valeur culminante atteinte par la statistique du moniteur sur cette même fenêtre.	Run par run. Le mécanisme d'alerte confronte directement cette variable au seuil λ .

ATTENTION. L'erreur e_t vaut 0 ou 1. Sur une exécution isolée, ce n'est donc pas une courbe : c'est une suite de sauts. Deux règles en découlent.

1. **$\tau_{\text{err}}(p)$ se mesure sur la moyenne inter-graines, jamais sur un run isolé.** Sur un run isolé il faudrait lisser, et un filtre de fenêtre W retarde le signal d'environ $W/2$ pas — du même ordre que le transitoire que vous cherchez à voir. Moyenne plutôt sur les graines, à chaque instant t . Avec 200 graines, l'incertitude sur la proportion d'erreur descend autour de $0,035$, ce qui suffit largement pour un saut d'amplitude comprise entre $0,10$ et $0,50$, et sans le moindre déphasage. Attention : cette précision suppose bien 200 graines. Sur la sous-grille de prototypage à 20 graines conseillée en annexe, elle remonte vers $0,11$, soit l'ordre de grandeur des plus petites amplitudes — ne concluez rien à ce stade. N'ajoutez aucun lissage temporel par-dessus : il déplacerait la date du drift.
2. **$A(H)$ se calcule run par run, et c'est lui qui sert à la question D.** $\tau_{\text{err}}(p)$ est une grandeur d'ensemble : elle ne produit pas un nombre par exécution, donc elle ne peut pas entrer dans une corrélation. $A(H) = \sum (e_t - \hat{p}_0)$ le produit, et se calcule sans aucun filtre. Retranchez bien la ligne de base pré-drift $\hat{p}_0$, mesurée avant le drift : sans elle, sur un horizon long, le bruit de fond pèse plus lourd que le choc lui-même. Faites varier H sur $50, 100, 200, 500$ — trop court, vous tronquez le rétablissement ; trop long, vous noyez le signal. La sensibilité à H est elle-même un résultat à rapporter.

Question D. Tracez sur un même axe temporel la fraction d'arbres remplacés, l'erreur globale de l'ensemble, et l'état interne du détecteur externe. Mesurez ensuite le lien entre τ_{ARF} et vos mesures alternatives par une [corrélation de rang](#), et non par une corrélation de Pearson.

Il y a en effet au moins trois raisons d'**écarter la mesure standard de corrélation (de Pearson)** :

1. La relation attendue est monotone mais pas linéaire — l'article mesure $\tau_{\text{ARF}} \approx 18,5 \cdot \Delta e^{(-0,98)}$, une loi de puissance, et Pearson sous-estimerait un lien pourtant fort.
2. Les distributions ont des queues lourdes, donc quelques exécutions extrêmes domineraient le calcul.
3. La présence de données **censurées** : quand aucun arbre n'est remplacé avant la fin de l'horizon, le temps exact d'adaptation est inconnu (strictement supérieur à l'horizon). Problème : le coefficient de Pearson exige des valeurs numériques exactes et n'est pas défini dans ces situations indéterminées. Une corrélation de rang se contente de l'ordre. Elle permet donc de traiter ces cas : il suffit d'attribuer à ces exécutions inachevées les derniers rangs (ex-aequo) — ce qui est légitime ici (l'horizon de censure est le même pour toutes les exécutions donc toute valeur censurée dépasse effectivement toute valeur observée). Gardez-les impérativement ; les supprimer pour forcer un calcul introduirait un biais de survie qui ruinerait l'analyse (l'erreur à éviter en [question A](#)).

Lequel des deux coefficients de rang ? Le **tau-b de Kendall** (`scipy.stats.kendalltau`, qui le calcule par défaut), pas le rho de Spearman. La raison tient à la façon dont chacun traite les ex-aequo — et vous en aurez beaucoup, de deux sortes.

1. **τ_{ARF} contre $\tau_{\text{swap}}(q)$ — ex-aequo des deux côtés.** Ce sont deux temps d'atteinte : si le quota d'arbres n'est jamais atteint, les deux variables sont censurées ensemble à l'horizon. Spearman leur donne à toutes deux le rang maximal, ce qui forme un bloc dans le coin haut-droit du nuage et tire le coefficient vers **+1** sans qu'aucune information n'ait été apportée. Le tau-b écarte ces paires de son numérateur et corrige son dénominateur en conséquence, tout en gardant l'ordre correct entre un run abouti et un run censuré.
2. **τ_{ARF} contre $A(H)$ ou $S_{\text{max}}(H)$ — ex-aequo d'un seul côté.** Ces deux grandeurs se calculent sur un horizon fixe : elles existent pour tous les runs et ne sont donc **jamais censurées**. Seul τ_{ARF} porte alors des ex-aequo. Le tau-b les retire de son numérateur, et sa correction de dénominateur ne compense que partiellement : le coefficient est tiré vers **0**. Attendez-vous donc à des valeurs faibles sur les petites amplitudes de drift, là où beaucoup d'arbres ne sont jamais remplacés. C'est un effet de la censure, pas une absence de lien.

*Conseil: Rapportez systématiquement le **taux de censure**, et refaites le calcul sur les seules exécutions complètes : si les deux valeurs divergent nettement, c'est votre censure qui pilote le résultat, pas le phénomène.*

A l'issue de vos expériences, vous aboutirez à l'une ou l'autre de ces situations:

1. **Le lien est fort** : l'indicateur de l'article est validé, la critique tombe, et vous fournissez la mesure qui manquait.
2. **Le lien est faible** : l'indicateur doit être remplacé, et vous fournissez celui qui le remplace.

Un résultat négatif clairement établi vaut autant qu'un résultat positif. Ne cherchez pas à confirmer l'article.

ATTENTION : dans les expériences actuelles, l'appel à `predict_one()` a été retiré pour isoler le mécanisme interne. Conséquence, **l'erreur de l'ensemble n'est jamais observée** dans ces scripts. Vous devrez construire votre propre instrumentation, pas réutiliser celle-là.

5. Ressources

Le code : [The-Blind-Spot-Paradox-Experiments](#) — scripts des expériences [R1](#) à [R9](#), résultats stockés, dépendances figées, tests. Chaque expérience est autonome et se relance en une commande.

Le manuscrit : [articleA_blindspot_v64_camera_ready.pdf](#), dans `docs/manuscript/` du dépôt. À lire dans cet ordre, et rien de plus pour démarrer.

Passage	Page	Contenu
Résumé	1	Le phénomène en dix lignes
Définition 2 — Adaptation Time & Blind Spot	2	La définition de τ_{ARF} que vous allez remettre en cause
§III-C — The Hydra Effect	3	$\tau_{\text{ARF}} = \min(\tau_1, \dots, \tau_M)$ et l'hypothèse d'indépendance
Proposition 3 — Starvation	3	Ce qu'un détecteur cumulatif peut voir d'un signal bref (partie corrigée en v64)
Figure 1	4	Remplacements internes et détections externes sur un axe commun
Proposition 9 et Corollaire 10	5	La course, et la taille critique d'ensemble
Figure 2	5	Les trois régimes : détecteur gagnant, alarme tardive, détection manquée
Table I	7	Échec complet de CUSUM avec ARF, récupération avec KSWIN
§V-B — Limitations	9	Les auteurs y reconnaissent eux-mêmes l'hypothèse d'indépendance

Le reste de l'article traite d'autres aspects. Vous pouvez l'ignorer.

Quelle expérience produit quoi

Chaque expérience se relance seule par `./run_experiment_R<n>.sh`, ou toutes d'un coup par `./run_all.sh`. Le tableau donne la correspondance entre le script, ce qu'il mesure, et l'endroit du manuscrit où le résultat apparaît.

Exp.	Script	Ce qu'elle mesure	Où ça sort dans l'article
R1	<code>exp_R1_generate_data.py</code>	Diagnostic de la course, balayage du seuil λ , alarmes tardives	§III, Figure 1
R2	<code>exp_R2_instrumented_blind_spot.py</code>	Le point aveugle instrumenté : remplacements internes vs alarmes	§IV, Figure 2
R3	<code>exp_R3_regime_crossover.py</code>	Bascule de régime, forêt statique vs adaptative, coût en précision	§IV
R4	<code>exp_R4_main_table.py</code>	Comparatif détecteurs × classifieurs sur ProteuS	Table I
R5	<code>run_experiment_R5.sh</code>	Évaluation sur données réelles, inondation de fausses alarmes	Table II
R6	<code>exp_R6_generate_data.py</code>	Arbre seul (HAT) de référence — donne l'accélération apportée par la forêt	§III-C
R7	<code>run_experiment_R7.sh</code>	Taux de détections manquées en fonction de l'amplitude du drift	§IV
R8	<code>exp_R8_lambda_op_sweep.py</code>	Seuil maximal admissible λ_{op} en fonction de l'amplitude	§V-A
R9	<code>exp_R9_compute_mcrit.py</code>	Taille critique d'ensemble M_{crit}	Corollaire 10

Par où commencer. R2 est votre point d'entrée : c'est la seule expérience déjà instrumentée sur les remplacements d'arbres, et c'est celle que vous allez étendre pour la [question D](#). Pour la [question B](#), R6 et R9 sont vos deux sources : R6 donne la loi du temps d'adaptation d'un **arbre seul**, R9 celle de la **forêt entière**. Comparer les deux lois, c'est déjà la moitié du travail théorique — c'est exactement l'écart que l'hypothèse d'indépendance prétend prédire.

Où sont les résultats. Chaque expérience écrit dans `results/R<n>_<nom>`, en Parquet ou CSV, avec ses figures. Des tests de non-régression existent pour R6 à R9 : ils vérifient que vos ré-exécutions retrouvent les valeurs publiées. Lancez-les avant de modifier quoi que ce soit, puis après.

Bibliothèque : [River](#), version 0.23.0 exactement ([Montiel et al., 2021](#)). Le compteur interne des remplacements d'arbres est accessible via `ARFClassifier._drift_tracker`.

6. Livrables et calendrier

Organisation suggérée : deux à trois personnes sur le versant théorique, deux à trois sur le versant expérimental, avec au moins un point de synchronisation au milieu. Les deux versants interagissent : les mesures orientent le modèle, le modèle dit quoi mesurer.

Semaine 1 — prise en main du dépôt et une expérience rejouée à l'identique ; formalisation écrite des questions A et B (quel est exactement l'objet aléatoire, quelles sont les hypothèses) ; première instrumentation sur une configuration simple.

Semaine 2 — campagne de mesures sur plusieurs amplitudes de drift et plusieurs graines ; démonstration de la question B, ou contre-exemple documenté ; rédaction.

À rendre : un rapport synthétique de 10 à 15 pages (15 pages est un maximum, 10 pages n'est pas un minimum: quelques pages suffisent si le rapport répond à tous les points du projet), un code source strictement reproductible en une commande unique, et une soutenance de 20 minutes.

Évaluation : clarté de la formalisation, reproductibilité du code, honnêteté sur ce qui n'a pas marché. Pas la quantité de résultats positifs.

Un résultat négatif rigoureusement établi possède la même valeur qu'une confirmation des travaux de l'article.

7. Glossaire

Concept drift

Changement, au cours du temps, de la relation entre les variables d'entrée et la variable à prédire — formellement, la loi conditionnelle $P(y | x)$ change ([définition générale](#)). Un modèle entraîné avant le changement devient faux après. Référence : [Gama et al. \(2014\)](#).

Data drift

Changement de la distribution des seules variables d'entrée, $P(x)$, sans que la relation à prédire soit nécessairement affectée — on parle aussi de [dataset shift](#). Un data drift n'entraîne pas forcément une perte de performance ; un [concept drift](#) si. Les deux se détectent avec des architectures différentes, comparées au §1. Référence : [Moreno-Torres et al. \(2012\)](#).

ARF (Adaptive Random Forest)

Forêt aléatoire conçue pour l'apprentissage en flux. Chaque arbre est entraîné en continu et possède son propre détecteur interne ([ADWIN](#)) qui surveille ses erreurs. Quand un arbre se dégrade, un arbre de remplacement entraîné en arrière-plan prend sa place. C'est ce **remplacement** qui rend la forêt capable de se réparer — et qui efface le signal que le détecteur externe attendait. Référence : [Gomes et al. \(2017\)](#).

ADWIN

Détecteur de changement à fenêtre adaptative. Il maintient une fenêtre de données récentes, la coupe en deux, et déclare un changement quand les moyennes des deux moitiés s'écartent trop. Il ne teste pas à chaque pas de temps mais tous les c pas : ce paramètre est son « horloge ». Dans l'[ARF](#), chaque arbre en possède un. Référence : [Bifet et Gavalda \(2007\)](#).

CUSUM / Page-Hinkley

Famille de détecteurs à **accumulation de preuve**. À chaque pas de temps on ajoute à un compteur l'excès d'erreur au-delà d'une tolérance δ_P , et on remet le compteur à zéro s'il devient négatif. L'alarme est levée quand le compteur dépasse un seuil λ . Conséquence directe : ces détecteurs ont besoin d'un signal **persistant**, et un signal fort mais bref ne les fait pas alarmer. C'est le cœur du problème étudié. Référence : [Page \(1954\)](#).

KSWIN

Détecteur qui compare la distribution d'une fenêtre récente à celle d'une fenêtre de référence, par un test de Kolmogorov-Smirnov. Il ne cumule pas de preuve dans le temps, ce qui le rend moins sensible aux signaux brefs — mais il échoue à son tour si le signal est plus court que sa fenêtre. Référence : [Raab, Heusinger et Schleif \(2020\)](#).

Instrumenter

Ajouter des points de mesure **à l'intérieur** d'un système pour observer directement son fonctionnement interne, au lieu de le déduire de son comportement extérieur. Ici : lire à chaque pas de temps le compteur interne de remplacements d'arbres de la forêt, plutôt que de simuler la course entre classifieur et détecteur avec deux processus indépendants.

C'est le standard académique sur ce sujet précis, pour une raison de fond. La plupart des comparaisons de détecteurs les évaluent sur des flux d'erreurs pré-générés, sans classifieur dans la boucle : l'interaction entre les deux devient invisible, et le phénomène étudié ici ne peut littéralement pas être observé. Instrumenter est ce qui le rend mesurable plutôt que simplement plausible — et réfutable, puisque chaque instant mesuré est vérifiable dans le code.

Hétéroscédasticité

Propriété d'une série dont la variance change au cours du temps. En finance, la volatilité se regroupe en paquets : des périodes calmes alternent avec des périodes agitées. Un détecteur qui suppose une variance constante interprète ces variations comme des drifts et produit des fausses alarmes en rafale.

ProteuS

Générateur de flux financiers synthétiques utilisé dans l'article. Il reproduit l'[hétéroscédasticité](#) et les changements de régime des séries de marché, avec des instants de rupture **connus**, ce qui permet de compter les détections manquées et les fausses alarmes. Référence : [Suárez-Cetrulo, Cervantes et Quintana \(2025\)](#).

Statistiques d'ordre

Branche des probabilités qui étudie la loi du minimum, du maximum et des valeurs classées d'un échantillon ([définition](#)). Si M variables indépendantes ont pour fonction de répartition F , le minimum a pour fonction de survie $(1 - F)^M$. Tout l'enjeu de la [question B](#) est de savoir ce que devient ce résultat quand l'indépendance tombe. Référence : [David et Nagaraja \(2003\)](#).

Risques concurrents et censure

Cadre statistique décrivant une situation où plusieurs événements sont en compétition et où le premier qui survient met fin à l'observation. La **censure** désigne le cas où l'observation s'arrête sans qu'aucun événement ne survienne — ici, une exécution où le détecteur n'alarme jamais. Cadre standard en analyse de survie ; l'estimateur non paramétrique de référence est celui d'Aalen-Johansen, [implémenté dans lifelines](#). Référence : [Fine et Gray \(1999\)](#).

Bibliographie

Les principales références sont fournies ci-dessous. Les autres sont dans la bibliographie du manuscrit.

- Gama, Žliobaitė, Bifet, Pechenizkiy, Bouchachia (2014). [A survey on concept drift adaptation](#). ACM Computing Surveys 46(4). — L'entrée en matière sur le [concept drift](#).
- Gomes, Bifet, Read et al. (2017). [Adaptive random forests for evolving data stream classification](#). Machine Learning 106. — Le classifieur étudié, [ARF](#).
- Montiel, Halford, Mastelini et al. (2021). [River: machine learning for streaming data in Python](#). JMLR 22. — La bibliothèque utilisée.
- Bifet, Gavaldà (2007). [Learning from time-changing data with adaptive windowing](#). SIAM SDM. — Le détecteur interne, [ADWIN](#).
- Page (1954). [Continuous inspection schemes](#). Biometrika 41. — L'origine de [CUSUM](#).

Compléments selon le versant que vous prenez :

- Moreno-Torres, Raeder, Alaiz-Rodríguez, Chawla, Herrera (2012). [A unifying view on dataset shift in classification](#). Pattern Recognition 45(1), 521–530. — La référence sur le [data drift](#) et sa taxonomie.
 - Suárez-Cetrulo, Cervantes, Quintana (2025). [ProteuS: A Generative Approach for Simulating Concept Drift in Financial Markets](#). arXiv:2509.11844. — Le générateur de flux financiers, [ProteuS](#).
 - Raab, Heusinger, Schleif (2020). [Reactive soft prototype computing for concept drift streams](#). Neurocomputing 416, 340–351. — [KSWIN](#).
 - Lu, Liu, Dong, Gu, Gama, Zhang (2018). [Learning under concept drift: A review](#). IEEE TKDE 31(12). — Revue générale du [concept drift](#), en complément de Gama et al.
 - David, Nagaraja (2003). *Order Statistics*, 3e édition. Wiley. — [Statistiques d'ordre](#).
 - Fine, Gray (1999). [A proportional hazards model for the subdistribution of a competing risk](#). Journal of the American Statistical Association 94(446), 496–509. — [Risques concurrents](#).
-

Annexe — Démarrage rapide

1. Cloner le dépôt et installer les dépendances. Ne mettez pas River à jour : le comportement interne de l'ARF a changé entre versions, et les résultats publiés ne se reproduisent qu'en 0.23.0.

```
git clone https://github.com/TheBlindSpot-ICDM2026/The-Blind-Spot-Paradox-Experiments.git
cd The-Blind-Spot-Paradox-Experiments
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt      # ou la commande indiquée dans le README

# Environnement de référence – à vérifier avant toute exécution
python -c "import sys, river, numpy; print(sys.version); print(river.__version__, numpy.__version__)"
# attendu : Python 3.12.x · river 0.23.0 · numpy 1.26.x

export PYTHONHASHSEED=0      # indispensable : sans cela vos résultats ne sont pas reproductibles
```

2. Rejouer l'expérience R2 telle quelle et vérifier que vous retrouvez les résultats stockés. Si ce n'est pas le cas, arrêtez-vous là et signalez-le : rien de ce qui suit n'aura de sens.

```
bash run_experiment_R2.sh
ls -l results/R2_instrumented_blind_spot/

# Les tests de non-régression comparent vos ré-exécutions aux valeurs publiées
pytest -q tests/
```

3. Copier le script de R2 et ajouter l'observation de l'erreur. Le script est `exp_R2_instrumented_blind_spot.py`. Ajoutez un appel à `predict_one()` avant chaque `learn_one()` et enregistrez l'erreur de l'ensemble à chaque pas. C'est la brique manquante de tout le versant expérimental.

```
cp experiments/R2_instrumented_blind_spot/exp_R2_instrumented_blind_spot.py \
  experiments/R2_instrumented_blind_spot/exp_S1_traces.py
```

4. Enregistrer en parallèle le compteur `_drift_tracker` et l'état interne du détecteur externe. Vous avez alors les trois trajectoires de la question D.

5. Étendre à plusieurs amplitudes de drift et plusieurs graines. Reprenez la grille de R8 — 21 amplitudes de 0,10 à 0,50, 200 graines. C'est le cadre sur lequel l'article publie ses courbes, donc celui qui rend vos résultats directement comparables aux siens.

ATTENTION AU COÛT DE CALCUL : Cette grille représente 4 200 exécutions, et réinsérer `predict_one()` double le temps d'exécution (chaque instance traverse les dix arbres une fois pour l'inférence et une fois pour l'apprentissage). Trois règles simples rendent ce calcul parfaitement fluide :

- Tronquez la queue post-drift :** R8 tourne sur 52 000 pas, dont 50 000 pas post-drift dédiés à sa propre analyse de censure extrême. Pour la question D, conservez impérativement la période d'apprentissage pré-drift (2 000 pas indispensables pour roder la forêt et estimer $\hat{p}_0$), mais limitez l'observation post-drift à 1 000 pas (ce qui couvre largement votre horizon $H = 500$). Vous divisez ainsi le temps de calcul par un facteur 17.
- Prototypiez sur sous-grille :** validez toute votre chaîne d'analyse sur 5 amplitudes × 20 graines, soit 100 runs, avant de lancer la campagne complète. Chronométrez ce lot : il vous donne directement le temps à prévoir pour les 4 200.
- Parallélisez sur les graines :** Les 200 graines étant strictement indépendantes, distribuez les calculs avec `joblib` ou `multiprocessing`.

ATTENTION AU PIÈGE D'API (ARFClassifier._drift_tracker) : Cet attribut interne est un dictionnaire cumulatif `{id_arbre: total_replacements}` (un **stock**), et non un flux d'événements. Ne faites jamais `cumul += sum(_drift_tracker.values())` dans votre boucle temporelle, sous peine d'intégrer le passé à chaque pas :

- Pour `τ_ARF` : détectez simplement le premier pas où `any(v > 0 for v in _drift_tracker.values())`.
- Pour `τ_swap(q)` : comptez la fraction d'arbres **distincts** renouvelés : `sum(1 for v in _drift_tracker.values() if v > 0) / n_models`. Si le même arbre est remplacé trois fois de suite, il ne compte que pour un seul arbre renouvelé dans la forêt.